

TECHNICAL DOCUMENTATION

NovaPay

Cross-border multi-currency digital banking platform for Africa-connected users, freelancers, diaspora senders, and businesses.

Version: 1.0

Date: 14 July 2026

Scope: Monorepo - NestJS API & Expo mobile (Phases 0-4 / F0-F7)

Classification: Internal project documentation

Source of truth: apps/api, apps/mobile, and docs/ phase specifications.

Contents

1. Introduction & Product Overview
2. Repository Structure
3. Technology Stack
4. Core Features & User Flows
5. API Architecture
6. Database Schema
7. Mobile Application
8. Configuration & Environment
9. Local Development Setup
10. Phased Delivery Roadmap
11. Security & Compliance Notes

1. Introduction & Product Overview

NovaPay is a cross-border, multi-currency digital banking platform built for Africa-connected users. It bridges local African financial rails (bank transfers and mobile money) with global currencies so people and businesses can hold, convert, send, receive, and spend money with transparent FX.

1.1 Purpose

- Hold balances in NGN, USD, and EUR
- Convert currencies with locked, transparent FX quotes
- Fund wallets and pay out to bank or mobile-money beneficiaries
- Issue virtual USD cards for online spend
- Receive inbound ACH/SEPA credits via virtual accounts
- Pay utility bills and buy airtime
- Deposit stablecoins and convert 1:1 to/from USD
- Operate business workspaces with roles and bulk payout approvals

1.2 Target Users

Segment	Needs
Individuals & freelancers	Receive USD/EUR from global clients, convert to NGN, spend on virtual cards
Diaspora senders	Move money from UK/EU/US to Africa with clear rates and payout rails
Businesses & organizations	Multi-currency ops, team roles, dual-authorization bulk payouts, public API

2. Repository Structure

NovaPay is a monorepo containing the backend API, the Expo mobile client, and phase documentation.

```
NovaPay/  
+-- apps/  
|   +-- api/           NestJS modular monolith (backend)  
|   +-- mobile/        Expo + React Native (frontend)  
+-- docs/  
|   +-- phase-0/ ... phase-4/ Backend phase specifications  
|   +-- frontend/      Mobile phase specifications (F0-F7)  
+-- docker-compose.yml  Local infrastructure (Redis)  
+-- .env.example        Environment template  
+-- README.md           Repository overview
```

3. Technology Stack

3.1 Backend (apps/api)

Layer	Choice
Framework	NestJS (TypeScript), modular monolith
ORM	Drizzle ORM
Database	PostgreSQL (Supabase hosting)
Cache / locks	Redis (ioredis)
Auth	Passport JWT (access + refresh tokens)
Validation	Zod (env), class-validator (DTOs)
Logging	Pino / nestjs-pino
Health	@nestjs/terminus
Cloud target	AWS (events via SNS/SQS or Kafka in later wiring)

3.2 Mobile (apps/mobile)

Layer	Choice
Framework	Expo SDK 54 + React Native + TypeScript
Routing	Expo Router (file-based, auth / tabs / modals)
Typography	Plus Jakarta Sans (UI), DM Sans (money/numbers)
Secure storage	expo-secure-store
Biometrics	expo-local-authentication
Motion	react-native-reanimated, gesture-handler

4. Core Features & User Flows

4.1 Authentication & Onboarding

1. Welcome carousel introduces product value propositions.
2. User enters phone number in E.164 format.
3. SMS OTP is issued (Redis-backed). Sandbox supports fixed code 000000.
4. User sets a 4-digit PIN for unlock and transaction authorization.
5. Optional biometric opt-in (Face ID / Touch ID).

4.2 KYC

Users submit identity documents (e.g. NIN, BVN, Passport). Sandbox can auto-approve submissions, moving users from tier_0 (restricted) to tier_1 (active). Payouts and card issuance require tier_1+; tier_0 single-payout limit is 0.

4.3 Wallets & Double-Entry Ledger

Multi-currency wallets (NGN, USD, EUR). Every financial movement posts balanced debit/credit rows in ledger_entries. Wallet balance is the sum of ledger entries for that wallet. Amounts are stored as bigint minor units (kobo/cents).

4.4 Funding

1. User selects Add Money on the NGN wallet and enters an amount.
2. Payment rail: Flutterwave (live NGN rails) or mock/sandbox.
3. Sandbox: simulate-complete credits the ledger immediately.

4.5 FX Conversion

1. User requests a quote (e.g. NGN <-> USD).
2. Rate = mid-market + configurable margin (basis points).
3. Quote locks for 60 seconds (TTL countdown in the app).
4. Booking executes a double-entry transfer between wallets.

4.6 Send & Payouts

Beneficiaries may be bank accounts or mobile money (MTN, Airtel, M-Pesa, etc.). Payouts require PIN authorization and are processed via Flutterwave or mock rails.

4.7 Virtual USD Cards

- Issue virtual Visa/Mastercard on the USD wallet.
- Full PANs are never stored - only last4, expiry, and processor references.
- Controls: freeze, unfreeze, close.
- Sandbox can simulate authorize -> settle against the USD ledger.

4.8 Receive, Bills, Airtime, Stablecoins & Business

- Virtual USD/EUR receiving accounts (routing, IBAN, BIC, bank name).
- Simulate inbound ACH/SEPA credits in sandbox.
- Biller catalogs and airtime operators by country, paid with PIN gate.
- Deposit USDC/USDT; convert 1:1 to/from fiat USD.
- Organizations with roles: owner, admin, approver, member.
- Bulk payout batches with dual authorization above approval limits.
- Developer API keys (np_test_...) and webhook registration.

5. API Architecture

The API is a NestJS modular monolith. Routes are versioned under /v1 and secured with JWT unless marked @Public().

5.1 Auth & Profile

Method	Endpoint	Notes
POST	/v1/auth/otp/request	Public - request OTP
POST	/v1/auth/otp/verify	Public - verify OTP, return JWTs
POST	/v1/auth/token/refresh	Public - refresh access token
GET	/v1/auth/me	Session hydrate (profile + KYC)
PATCH	/v1/auth/profile	Update profile
POST	/v1/auth/kyc	Submit KYC documents

5.2 Wallets, Payments & Transactions

Method	Endpoint	Notes
GET	/v1/wallets	List wallets & balances
GET	/v1/payments/rail	Active rail (mock / flutterwave)
GET	/v1/payments/banks/ng	Nigerian bank list
GET/POST	/v1/beneficiaries	List / add beneficiaries
POST	/v1/payments/fund	Start wallet funding
POST	/v1/payments/fund/:id/simulate-complete	Sandbox credit
POST	/v1/payments/payout	Payout to beneficiary
GET	/v1/transactions	Paginated history
GET	/v1/transactions/:id	Transaction detail
POST	/v1/payments/flutterwave/webhook	Public webhook

5.3 FX & Cards

Method	Endpoint	Notes
GET	/v1/fx/rate	Live rates
POST	/v1/fx/quotes	Create locked quote
POST	/v1/fx/quotes/:id/book	Book quote / convert
POST/GET	/v1/cards	Issue / list cards
POST	/v1/cards/:id/freeze unfreeze close	Card controls
POST	/v1/cards/:id/authorize	Sandbox auth hold
POST	/v1/cards/authorizations/:authId/settle	Settle auth

5.4 Receive, Bills, Stablecoins

Method	Endpoint	Notes
POST/GET	/v1/accounts/virtual	Create / list virtual accounts
POST	/v1/accounts/virtual/:id/simulate-inbound	Sandbox inbound

GET	/v1/bills/catalog	Billers by country
GET	/v1/airtime/operators	Operators by country
POST	/v1/bills/pay	Pay bill
POST	/v1/airtime/topup	Airtime top-up
GET	/v1/stablecoins/balances	Stablecoin balances
POST	/v1/stablecoins/deposit convert	Deposit / convert

5.5 Business, Public API & Health

Method	Endpoint	Notes
POST/GET	/v1/business/organizations	Create / list orgs
POST/GET	/v1/business/organizations/:orgId/members	Invite / list members
GET/POST	/v1/business/.../wallet	Org wallet & fund
POST	/v1/business/.../bulk-payouts	Submit / approve batches
POST/GET	/v1/developer/api-keys	Create / list keys
POST	/v1/developer/api-keys/:id/revoke	Revoke key
POST/GET	/v1/developer/webhooks	Register / list webhooks
GET	/v1/public/wallets transactions	API-key gated public read
GET	/health	Postgres, Redis, app checks
GET	/health/live	Liveness probe

6. Database Schema

PostgreSQL via Drizzle ORM. Monetary fields use bigint minor units to avoid floating-point error.

6.1 Important Enums

Enum	Values (summary)
user_status	pending, active, suspended, closed
wallet_owner_type	user, system, organization
wallet_status	active, frozen, closed
ledger_direction	debit, credit
transaction_type	fund, payout, transfer, convert, card_spend, receive, bill, airtime, stablecoin_*, bulk_payout, reversal, adjustment
transaction_status	pending, processing, completed, failed, reversed
beneficiary_type	bank, mobile_money
kyc_status	pending, approved, rejected
card_status	active, frozen, closed
fx_quote_status	open, booked, expired
org_role	owner, admin, approver, member
bulk_payout_status	draft, pending_approval, processing, completed, failed, rejected

6.2 Core Tables

Table	Role
users	Phone, email, tag, status, KYC tier
wallets	Per-owner, per-currency fiat wallets
transactions	High-level money movement records + idempotency
ledger_entries	Double-entry rows; balance = sum by wallet
beneficiaries	Saved bank / MoMo recipients
kyc_records	Document verification trail
fx_quotes	Locked quotes with mid/client rate & TTL
cards	Virtual/physical cards (no full PAN)
virtual_accounts	Inbound receive account details
organizations	Business profiles + approval limits
organization_members	User <-> org role mapping
bulk_payout_batches / items	Org payout batches and line items
api_keys	Hashed developer keys + scopes
webhook_endpoints	Developer webhook subscriptions

Ledger invariant: every completed financial action must produce balanced debit and credit ledger entries. The wallet balance is derived from the ledger, not stored as a mutable single column.

7. Mobile Application

File-based routing lives under apps/mobile/app.

7.1 Auth & Onboarding Routes

Screen	Purpose
(auth)/welcome	Onboarding carousel
(auth)/phone	E.164 phone input
(auth)/otp	OTP verification
(auth)/pin-setup	Set 4-digit PIN
(auth)/pin-unlock	Cold-start unlock gate
(auth)/kyc	Document submission

7.2 Main Tabs

Tab	Purpose
Home	Wallet balances, currency toggle, quick actions, recent activity
Cards	Virtual cards list + USD balance
Send	Beneficiaries and send entry points
Activity	Full transaction history with filters
Profile	Profile, KYC tier, money tools links

7.3 Action Screens

Fund, Convert, Add Beneficiary, Send Money, Create Card, Card Details, Bills, Receive, Stablecoins, Business workspace, Business org detail, Edit Profile, Change PIN.

8. Configuration & Environment

8.1 API (apps/api/.env)

```
NODE_ENV=development
PORT=3000
APP_NAME=novapay-api
APP_URL=http://localhost:3000

# Supabase Postgres
# Transaction pooler (6543) for runtime
DATABASE_URL=postgresql://...@...pooler.supabase.com:6543/postgres?sslmode=require
# Direct / session (5432) for migrations
DATABASE_URL_DIRECT=postgresql://...@...:5432/postgres?sslmode=require

REDIS_URL=redis://localhost:6379
JWT_ACCESS_SECRET=... # >= 32 chars
JWT_REFRESH_SECRET=... # >= 32 chars
JWT_ACCESS_TTL=15m
JWT_REFRESH_TTL=30d

OTP_DEV_FIXED_CODE=000000
KYC_AUTO_APPROVE=true
PAYMENT_RAIL=auto
FX_USD_NGN_MID=1600
FX_MARGIN_BPS=150
FX_QUOTE_TTL_SECONDS=60
CARD_ISSUER=mock
```

8.2 Mobile

Set EXPO_PUBLIC_API_URL to the API base URL.

Physical devices cannot use localhost - use your machine's LAN IP, e.g. <http://192.168.x.x:3000>.

8.3 Supabase Notes

- Runtime uses the transaction pooler URL (port 6543).
- Drizzle migrations use the direct/session URL (port 5432).
- SSL (sslmode=require) is required.
- Supabase is used as Postgres hosting only; auth/KYC remain in NestJS.

9. Local Development Setup

9.1 Infrastructure

```
docker compose up -d redis
```

9.2 API

```
cd apps/api
npm install
cp ../../.env.example .env # then fill Supabase + secrets
npm run db:push
npm run start:dev
# Health: http://localhost:3000/health
```

9.3 Mobile

```
cd apps/mobile
npm install

# Simulator
EXPO_PUBLIC_API_URL=http://localhost:3000 npx expo start

# Physical device (use LAN IP)
EXPO_PUBLIC_API_URL=http://192.168.1.50:3000 npx expo start --offline
```

Open with iOS/Android simulator, or scan the QR code in Expo Go.

10. Phased Delivery Roadmap

10.1 Backend Phases

Phase	Focus
0	Scaffold NestJS, Supabase Postgres, Redis; partners & regulatory notes
1	OTP auth, KYC, NGN double-entry ledger, fund & payout (mock/Flutterwave)
2	FX quotes, NGN <-> USD conversion, virtual USD cards
3	Virtual receive accounts, MoMo payouts, bills & airtime sandbox
4	Organizations, bulk payouts, stablecoins, public API keys & webhooks

10.2 Frontend Phases

Phase	Focus
F0	Design system, typography, tab shell
F1	Welcome, OTP, PIN lock, KYC
F2	Home wallets, fund modal, activity
F3	FX convert + send / beneficiaries
F4	Virtual cards + controls + simulated spend
F5	Bills, airtime, receive accounts
F6	Stablecoin deposit & convert
F7	Business orgs, invites, bulk payout approvals

Detailed specifications live in docs/phase-* and docs/frontend/phase-f*.

11. Security & Compliance Notes

- JWT access tokens are short-lived; refresh tokens rotate over a longer TTL.
- Session material and PIN unlock are protected on-device via SecureStore and optional biometrics.
- Card PANs are not persisted in the application database.
- API keys are stored hashed; only prefixes are returned after creation.
- Payment and BaaS webhooks are public endpoints and must validate provider signatures/hashes.
- KYC tiers gate high-risk actions (payouts, card issuance).
- Idempotency keys on critical money-moving endpoints prevent duplicate posts.
- Regulatory pathways and partner shortlists are documented under docs/phase-0/ (REGULATORY.md, PARTNERS.md).

NovaPay Project Documentation v1.0 - Generated 14 July 2026. This document summarizes the implemented monorepo for onboarding, architecture review, and delivery tracking.